Graph Algorithms

Readings

Chapter 22 of CLR

Appendix B of CLR, section B.4 on Graphs

The exercises in that section are good

Topics:

- Graph Data Structure
- Graph presentation
- BFS
- DFS
- Topological Sorting
- Connected Components

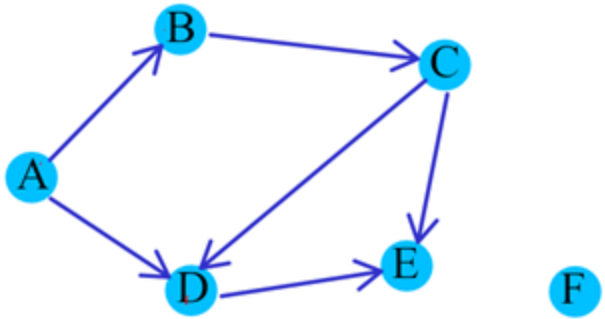
Graph

- A ***graph*** $G = (V, E)$ is an ordered pair consisting of
  - a set V of ***vertices*** (singular: ***vertex***),
  - a set $E \subseteq V \times V$ of ***edges***.

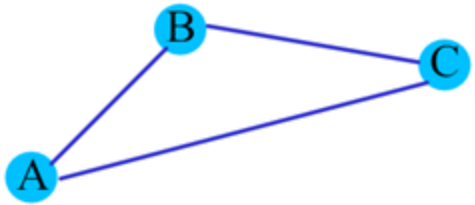
- E can be a set of ordered pair or unordered pairs
  - $G = (V, E)$ is ***directed graph*** if E consists of *ordered* pairs of vertices.
  - $G = (V, E)$ is an ***undirected graph*** if E consists of *unordered* pairs of vertices.
  - Number of vertices: $|V|$
  - Number of edges: $|E|$

Graph Example

- Here is a graph G = (V, E)
  - Each edge is a pair (v_1, v_2), where v_1, v_2 are vertices in V



V = {A, B, C, D, E, F}
E = {(A,B),(A,D),(B,C),(C,D),(C,E),(D,E)}



V = {A, B, C}
E = {(A,B),(A,C),(B,C)}

Undirected Graph: Terminology

- u and v are **adjacent** in an undirected graph G if (u,v) is an edge in G
  - edge e = (u,v) is **incident** with vertex u and vertex v
- **degree** of a node: ***deg(v)***: the number of edges incident to v

Directed Graph

- Vertex u is **adjacent to** vertex v in a directed graph G if (u,v) is an edge in G
  - vertex u is the **initial** vertex of (u,v)
- Vertex v is **adjacent from** vertex u
  - vertex v is the terminal (or end) vertex of (u,v)
- the ***indegree*** of a node v, ***indeg(v),*** is the number of edges entering v
- the ***outdegree*** of a node v, ***outdeg(v)***, is the number of edges leaving v
- deg(u) = indeg(u) + outdeg(u)
- **Source**: a node **u** is a source if indeg(u) = 0
- **Sink**: a node **u** is a sink if outdeg(u) = 0

Terminologies

- **A Path of length k**:
  - In the graph $G=(V,E)$, a path from vertex **u** to vertex **v** is a sequence $(v_0,v_1,\cdots,v_k)$ of vertices such that $u = v_0$, $v = v_k$, and $(v_{i-1},v_i) \in E$ for all i = 1,2,...,k. The length of the path is the number of edges in the path.
- The vertice v is **reachable** from u if there is a path from u to v
- **Simple path:** A path in which all the vertices are distinct
- **Cycle:** A special type of path where starting and ending vertices are the same $(v_0,v_1,\cdots,v_k)$ forms a cycle if $v_0 = v_k$, and the path contains at least one edge.
- A cycle is a special type of path starting and ending at the same vertex.
- A graph with no cycles is **acyclic**.
- **Subgraph**: graph ***H = (V', E')*** is subgraph of G = (V, E) if V' is subset of V, and E' is a subset of E.

Graph properties

- A graph is called **dense** if $E = \Theta(V^2)$
  - Most pairs of vertices are connected by an edge
- A graph is called **sparse** if it is not dense: $E \ll V^2$
  - There are very few edges in the graph
- **connected graph**: an undirected graph in which every vertex is reachable from all other vertices
- In an undirected graph, if the graph is connected:
  - There must be at least $|V| - 1$ edges $\rightarrow |E| \geq |V| - 1$
  - Proof by induction on the number of vertices
- $E = \Theta(V^2)$ and $|E| \geq |V| - 1 \rightarrow \log |E| = \Theta(\log V)$
- **Strongly connected graph**: a *directed* graph in which every two vertices are reachable from each other

T: $|E| \geq |V| - 1$ in an undirected connected graph

proof: $|V| = 1$ single vertex $0 \geq 1 - 1 = 0$

induction hypothesis: T is true for $|V| = n - 1$

induction step: need to prove T is true for $|V| = n$

in the graph with $|V| = n$, pick one vertex, say u

Since graph is connected there is at least one edge connecting u to the rest of the graph. remove that edge. In the new subgraph, $G' = (V', E')$

$|E'| \geq |V'| - 1$ ← True by IH

$1 + |E'| \geq |V'| - 1 + 1$ ← add 1 to both sides

$|E| \geq |V| - 1$

Graph

- **Tree:**
  - A connected (undirected) graph without any cycle
  - Tree is a graph with exactly one path between any pair of vertices
  - Theorem: a tree is a connected graph with `|V|-1` edges, i.e., `|E|=|V|-1`

  - To prove the above theorem, we prove the following two:
    - If a graph is a tree ---> |E| = |V| - 1
      - Proof by induction
    - If a graph is connected and it has |E| = |V| - 1 edges ---> it is a tree
      - Proof by contradiction

a tree is a connected graph with $|V|-1$ edges; $|E|=|V|-1$

(1) if a graph is tree $\longrightarrow$ $|E|=|V|-1$

proof by induction: Base Case: True for single vertex

IH: for every tree with $< n$ $\longrightarrow$ $|E|=|V|-1$

IS: consider a tree with n vertices. remove any edge

$\longrightarrow$ two trees with $< n$ vertices $|E_1|=|V_1|-1$

$|E_2|=|V_2|-1$

$|E|=|V|-1$ $\leftarrow$ $|E_1|+|E_2|=|V_1|+|V_2|-2$

$+1$ $+1$

a tree is a connected graph with $|V|-1$ edges; $|E|=|V|-1$

(2) if a connected graph has $|V|-1$ edges ⟶ it is a tree

↳ we show this graph has no cycle → it's a tree

proof by contradiction → suppose the graph has a cyle.

remove an edge from the cycle ⟶ graph still connected

in the new graph, $G' = (E', V')$ ⟶ $|E'| = |E| - 1$

$|V'| = |V|$

Since G' is connected $|E'| \geq |V'| - 1$ ⟶ $|E| - 1 \geq |V| - 1$ contradiction

Storing Graphs

There are two ways to store a graph:

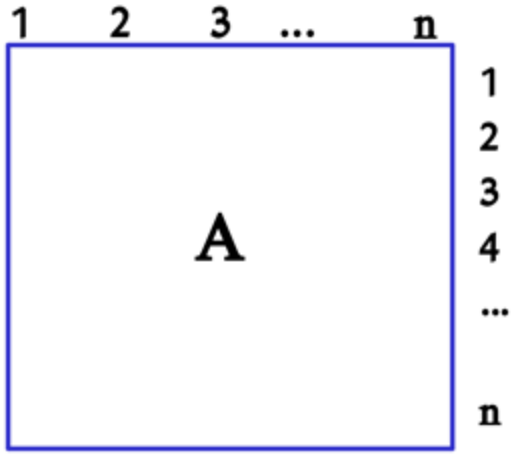
- Adjacency matrix
- Adjacency list

Storing Graphs: Adjacency-matrix

The adjacency matrix of a graph G = (V, E)

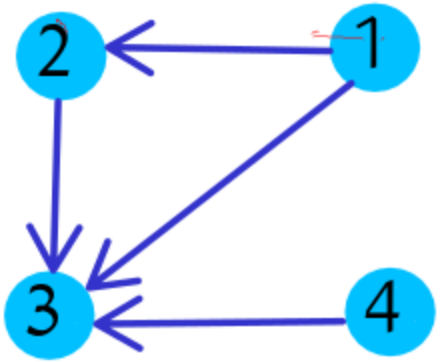
- V = {1, 2, …, n} is the set of vertices of the graphs
- is an **n** x **n** matrix A

$$A[i,j] = \begin{cases} 0 & \text{if } (i,j) \notin \\ \text{E} & \text{if } (i,j) \in \text{E} \end{cases}$$



- Space : $O(n^2)$

Storing Graphs: Adjacency-matrix: Example



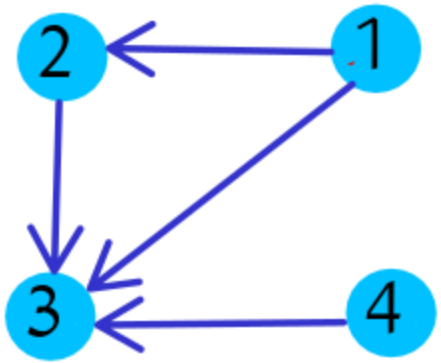
A	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	0	0	1	0

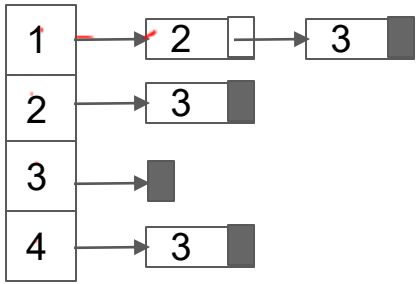
$\Theta(V^2)$ storage

Storing Graphs: Adjacency list

- An adjacency list of a vertex **v ∈ V** is the list **Adj[v]** of vertices adjacent to v
- For undirected graphs, **|Adj[v]| = degree(v)**
- For directed graphs, **|Adj[v]| = out-degree(v)**
- adjacency lists use **Θ(V + E)** storage
- **Handshaking Lemma:** for undirected graphs $\sum_{v \in V} deg(v) = 2|E|$

$$Adj[1] = \{2, 3\}$$
$$Adj[2] = \{3\}$$
$$Adj[3] = \{\}$$
$$Adj[4] = \{3\}$$





Storing Graphs

- Adjacency Matrix vs Adjacency List

	Adjacency matrix	Adjacency list
$(u, v) \in E$	$\Theta(1)$	$O(\deg(u))$
Time to list u's neighbor	$\Theta(n)$	$\Theta(\deg(u))$
Time to list all edges	$\Theta(n^2)$	$\Theta(n+m)$
Space complexity	$\Theta(n^2)$	$\Theta(n+m)$

Storing Graphs

Adjacency Matrix

- Advantage:
  - O(1) test for presence or absence of edges
- Disadvantage:
  - Inefficient for sparse graphs
  - Storage not efficient
  - Accessing edges not efficient

Adjacency List

- Advantage:
  - Good for sparse graphs
  - Accessing edges are easy
- Disadvantage:
  - More complex data structure
  - Not possible to access an edge in O(1)